

AnalyzerService: независимый от транспорта хост

Техническое руководство по версии проекта от 05.10.2026. Жизненный цикл DLL, последовательность вызовов, работа Sysmex CS-2000i, остановка, исключения, расширение и публикация для Windows Service.

1. Главный принцип архитектуры

Служба не решает, каким способом общается прибор. Она читает конфигурацию, создаёт файловый логгер, загружает управляемую DLL, передаёт ей настройки и командует работать. DLL владеет своим соединением, протоколом и рабочими потоками. Текущий Sysmex-драйвер выбирает TCP; другой плагин может читать COM-порт или папку обмена без изменения Host.

Windows Service / Worker -> AnalyzerManager -> AnalyzerRuntime
AnalyzerRuntime -> IAnalyzerDriver: Initialize -> RunAsync -> Stop -> Dispose
Sysmex DLL -> AnalyzerService.Transport.TcpHost -> TCP client / Host Online
Sysmex DLL -> LisDBProvider + RawResultQueue -> SQL / .raw / .res / .ok
Будущий COM/File DLL -> собственный порт или каталог; тот же IAnalyzerDriver

Проект	Ответственность	Что он не знает
AnalyzerService.Host	Windows Service, JSON, логирование, загрузка DLL, жизненный цикл	TCP, COM, Host Online, SQL-формат прибора
AnalyzerService.Contracts	IAnalyzerDriver, IAnalyzerLogger, AnalyzerSettings	Конкретный транспорт
SysmexCS2000.HostOnline.Driver	TCP-сеансы, R221/S221/D121/D221, очередь и ЛИС	Жизненный цикл Windows Service
AnalyzerService.Transport	Повторно используемый TCP listener, сеанс и статус сокета	Sysmex Host Online и Windows Service
LisDatabase / ResultFiles	Запросы ЛИС / накопление и обработка сырых файлов	Оркестрация всех приборов

2. Контракт DLL и смысл методов

Вызов	Кто вызывает	Ожидаемое поведение
Initialize(logger, settings)	AnalyzerRuntime	Синхронная проверка собственных полей JSON, создание объектов; I/O не ожидается.
RunAsync(token)	AnalyzerRuntime	Драйвер запускает ресурсы и сразу возвращает Task всего рабочего цикла; Task завершается при остановке или ошибке.
Stop()	AnalyzerRuntime	Синхронный сигнал: закрыть/разблокировать ожидание. Длительное ожидание здесь недопустимо.
Dispose()	LoadedDriver	Освободить ресурсы; затем AssemblyLoadContext помечается к выгрузке.

RunAsync возвращает Task потому, что сеть или иной внешний источник могут долго ждать событие. Хост не вызывает Task.Run вокруг драйвера: плагин сам выбирает эффективную модель. Для синхронного COM API драйвер может организовать отдельный поток и закрыть порт в Stop; для файлового обмена - наблюдатель каталога. SQL-запросы и короткие записи .raw в текущем Sysmex-драйвере остаются синхронными. Рабочая задача нужна хосту также для наблюдения ошибок.

3. Запуск: точная последовательность вызовов

1. Program создаёт Generic Host с Windows Service; Worker.ExecuteAsync ищет JSON только в каталоге configs рядом с EXE и читает каждый верхнеуровневый файл через JsonAnalyzerSettingsProvider.LoadAll.
2. AnalyzerSettingsValidator.ValidateAndThrow проверяет только поля хоста: AnalyzerName, Isdll и DllPath. Он не отвергает File/Serial и не проверяет IP/порт. Активные конфигурации передаются AnalyzerManager.Add; повторное имя запрещено.
3. AnalyzerManager.StartAllAsync последовательно вызывает AnalyzerRuntime.StartAsync для каждого активного прибора. Runtime создаёт отдельный файловый логгер и связанную отмену.

4. DriverLoader.Load разрешает DllPath относительно каталога EXE, создаёт DriverLoadContext и через reflection экземпляр IAnalyzerDriver. Только Contracts разделяется с Host; прочие зависимости ищутся рядом с DLL.
5. Runtime вызывает DLL.Initialize(logger, settings), затем DLL.RunAsync(token), сохраняет возвращённый Task как Completion и немедленно возвращается менеджеру. Sysmex-драйвер в Initialize проверяет Protocol/TCP/IP/порт/папки/БД.
6. Sysmex.RunAsync запускает RawResultQueue, создаёт TcpHost и синхронно открывает listener. Затем RunConnectionsAsync асинхронно ждёт подключения. Manager ожидает Task.WhenAll(Completion): все приборы могут работать одновременно.

```
Worker.ExecuteAsync
-> JsonAnalyzerSettingsProvider.LoadAll -> AnalyzerSettingsValidator
-> AnalyzerManager.Add -> AnalyzerManager.StartAllAsync
-> AnalyzerRuntime.StartAsync -> DriverLoader.Load
-> IAnalyzerDriver.Initialize -> IAnalyzerDriver.RunAsync
-> Sysmex.RunConnectionsAsync -> TcpHost.AcceptAsync
```

4. Пример рабочего стека: один запрос R221

1. TcpHost.AcceptAsync принимает TCP-клиента и сохраняет удалённый адрес; Sysmex.RunConnectionsAsync передаёт поток в HandleConnectionAsync.
2. ReadTextAsync ожидает STX, затем не более 253 ASCII-байт тела и ETX. В Protocol фиксируется точный RX; TcpHost.RecordRead обновляет состояние сокета. AddBlock объединяет многоблочный текст, если он получен.
3. HostOnlineCodec.ParseInquiry выделяет Sample ID, штатив, позицию и коды прибора. LisDBProvider.GetOrder синхронно читает пациента и незавершённые исследования из SQL; по каждому кандидату пишет причину включения или исключения.
4. BuildOrder преобразует код результата, например 392, в код группы задания 390, формирует S221 с полем пациента до 15 ASCII-символов. Число блоков ограничено 22 группами на текст. WriteTextAsync отправляет STX/S221/ETX и пишет точный TX.

В этом пути AnalyzerRuntime вообще не вызывается между Акцепт и TX: он только наблюдает Task рабочего цикла. Документированный формат R221/S221 взят из 2_5197664667366890337.pdf. ACK/NAK здесь не добавлены: этот вопрос был отложен до отдельной проверки с прибором.

5. Пример рабочего стека: D121 и результат ЛИС

1. HandleConnectionAsync получает D121/D221. Точный RX, тип сообщения, штатив, позиция, образец, коды, значения и флаги записываются в Protocol. Для контроля качества исходные байты сохраняются в Results/QualityControl без преобразования.
2. Для пациента RawResultQueue.SaveResult сначала атомарно пишет исходные STX/ETX-байты в отдельный .raw. После этого TCP-приёмник может читать следующее сообщение, не ожидая SQL и построения файлов ЛИС.
3. Выделенный поток RawResultQueue подбирает готовые .raw, вызывает ProcessStoredResult -> HostOnlineCodec.ParseResult -> HostOnlineResultHandler.Handle. Кодек восстанавливает десятичный масштаб по таблице кодов PDF; обработчик округляет числовой выход до одного знака и записывает запятую.
4. LisDBProvider.TranslateResultCode ищет код PSMV2. После успешной записи .res и .ok исходный .raw переносится в Results/archive; пустое сообщение, отсутствие сопоставленных тестов или ошибка - в Results/errors с записью в Error.

Пример: сырое 392 286 означает 28.6 и в выходном файле ЛИС записывается 28,6. Код 39100781 соответствует 0.781 до округления. Масштабы кодов взяты из таблицы 2_5197664667366890338.pdf, стр. 47/53; реальные Assay Group Settings прибора необходимо поддерживать согласованными с этой таблицей.

6. Остановка и освобождение ресурсов

1. Service Control Manager инициирует Worker.StopAsync; тот вызывает AnalyzerManager.StopAllAsync. Менеджер обходит все драйверы и собирает ошибки каждого, не оставляя остальные без команды остановки.
2. AnalyzerRuntime.StopAsync отменяет token, вызывает DLL.Stop и затем ожидает сохранённый Completion с ограничением cancellationToken. Хост не знает, какой порт, сокет или наблюдатель каталога остановил плагин.
3. В Sysmex Stop закрывает TcpHost: AcceptAsync и активное чтение клиента разблокируются. RunConnectionsAsync выходит и в finally останавливает поток RawResultQueue после текущего файла. Необработанные .raw остаются до нового старта.
4. AnalyzerManager вызывает AnalyzerRuntime.Dispose, тот освобождает LoadedDriver; DLL.Dispose закрывает остаточные ресурсы, DriverLoadContext.Unload помечает изолированную сборку к выгрузке. Затем освобождаются токены Runtime.

Если RunAsync завершился с ошибкой, Worker получает её через Completion, пишет файловый Error-журнал и передаёт Generic Host. Ошибки Stop, ожидания и Dispose тоже логируются; несколько ошибок агрегируются в AggregateException. Штатная отмена рабочей задачи при остановке ошибкой не считается; отмена лимита ожидания Stop фиксируется как ошибка.

7. Конфигурация и размещение DLL

Структура configs/SysmexCS2000.json сохранена. Хост читает общий набор полей, но интерпретирует лишь имя, активность и путь DLL. Поля ConnectionType, IPAddress, Port, Protocol, ConnectionString, ResultsFolder и OutputFolder принадлежат конкретному драйверу. Неактивный JSON в configs/examples не запускается, так как LoadAll просматривает только верхний уровень configs.

```
publish/AnalyzerService/AnalyzerService.Host.exe один исполняемый Host
publish/AnalyzerService/configs/SysmexCS2000.json
publish/AnalyzerService/drivers/SysmexCS2000HostOnline/
SysmexCS2000.HostOnline.Driver.dll + .deps.json
AnalyzerService.Transport.dll + LisDatabase.dll + ResultFiles.dll
Microsoft.Data.SqlClient.dll и остальные зависимости публикации
```

Только AnalyzerService.Contracts является общей сборкой с Host. Для нового драйвера создаётся своя подпапка drivers, туда помещаются его DLL, .deps.json и частные зависимости. DllPath указывает на главную DLL. Если драйвер использует TCP, он может сослаться на AnalyzerService.Transport; COM/File-драйверу это не требуется. Никаких новых полей JSON для этого рефакторинга не добавлено.

8. Сборка и проверки

1. Откройте AnalyzerService.sln в Visual Studio 2022 с .NET 9 SDK; выполните Restore NuGet и Build Solution. Для готовой публикации запустите package.ps1 из корня SysmexCS2000Ready.
2. Скрипт публикует framework-dependent однофайловый Host для win-x64; на машине службы нужен .NET 9 Runtime. DLL-плагины остаются внешними для динамической загрузки. Копируйте весь каталог publish/AnalyzerService, не только EXE или главную DLL.
3. Проверьте IPAddress для Sysmex: это должен быть локальный адрес сервера. Учётной записи службы нужны права чтения configs/drivers, записи в ResultsFolder, OutputFolder и Logs, а также доступ к SQL Server.

```
dotnet restore AnalyzerService.sln
dotnet build AnalyzerService.sln --no-restore --maxcpucount:1
dotnet run --project tests/SysmexCS2000.HostOnline.Driver.Tests --no-build
./package.ps1
./tests/PublishedPackageSmoke.ps1 # loopback, без реальной БД/прибора
```

Тесты проверяют в том числе DLL файлового обмена без IP/порта, отсутствие ссылок Host на транспорт/БД, Sysmex TCP-сеанс, точные QC-байты, формат чисел, журналы и остановку. Опубликованный пакет отдельно проверен на loopback. Проверка на стенде по-прежнему нужна для поведения реального анализатора и ЛИС.